

Machine Learning Applications in VLSI Testing

Million Solomon

dept. Electrical and Computer Engineering

Toronto Metropolitan University

Toronto, Canada

million.solomon@torontomu.ca

Abstract—The continuous advancement in VLSI (Very-Large-Scale Integration) technology necessitates robust and efficient testing methodologies to ensure the reliability and functionality of integrated circuits (ICs). Traditional testing techniques, while effective, often face challenges in coping with the increasing complexity and miniaturization of modern circuits. This paper explores the application of machine learning (ML) in VLSI testing, aiming to elucidate the synergy between these two domains and demonstrate the practical benefits of ML in IC testing. Initially, the concepts of VLSI testing and machine learning are introduced separately, establishing a foundational understanding of their principles and methodologies. Subsequently, the paper delves into various ML applications in VLSI testing, including fault detection, test pattern generation, and behavioral testing. By reviewing mainstream examples, the paper highlights the effectiveness of ML in enhancing test accuracy, reducing test time, and adapting to new testing challenges. A practical implementation is presented to demonstrate functional testing using ML, specifically through an 8-bit even parity generator. Utilizing the PyCaret library, multiple classifiers were evaluated to predict the parity from 8-bit inputs. The Ridge Classifier and Linear Discriminant Analysis both achieved an accuracy of 47.58%, while a neural artificial neural network (ANN) outperformed other models with an accuracy of 63.89%. These results underscore the potential of ML techniques to improve VLSI testing processes significantly. This paper aims to provide insights into the integration of ML in VLSI testing, showcasing how ML can enhance the reliability and efficiency of IC testing and paving the way for future research and development in this interdisciplinary field.

Index Terms—VLSI Testing, Machine Learning, Fault Detection, Test Pattern Generation, Design Verification, Parity Generator, PyCaret, Ridge Classifier, Linear Discriminant Analysis, Classification Accuracy, Artificial Neural Networks (ANN).

I. INTRODUCTION

Very-Large-Scale Integration (VLSI) technology refers to the process of creating integrated circuits (ICs) by integrating thousands to millions of transistors onto a single chip. This technological advancement has been instrumental in the development of modern electronic devices, enabling the production of compact, efficient, and powerful systems such as microprocessors, memory chips, and application-specific integrated circuits (ASICs). The complexity of VLSI circuits necessitates rigorous testing to ensure their functionality, reliability, and performance. VLSI testing involves verifying that each circuit component operates correctly and that the entire system meets its design specifications. This testing process is critical as it helps identify manufacturing defects, design errors, and other issues that could compromise the circuit's performance [1].

VLSI testing is crucial for ensuring the reliability, functionality, and performance of integrated circuits, which are foundational components of virtually all modern electronic devices. The importance of VLSI testing lies in its ability to detect and correct manufacturing defects, design errors, and other anomalies that could lead to circuit failures, thus preventing costly recalls and ensuring product quality and longevity [2]. Effective testing processes are essential to meet stringent industry standards and customer expectations, thereby enhancing the overall market competitiveness of semiconductor companies. However, VLSI testing poses significant challenges due to the increasing complexity and miniaturization of circuits. As the number of transistors on a chip grows, the complexity of the testing process increases exponentially, requiring advanced methodologies to maintain efficiency and thoroughness [3]. Additionally, the high costs associated with testing, the need for rapid time-to-market, and the intricate nature of diagnosing and localizing faults within dense circuitry further complicate the testing process. Addressing these challenges necessitates innovative approaches, such as leveraging machine learning techniques, to enhance test coverage, reduce costs, and improve diagnostic accuracy.

The objective of using machine learning in VLSI testing is to address the significant challenges posed by traditional testing methodologies through the application of intelligent and adaptive techniques. Machine learning algorithms can analyze vast amounts of test data to identify patterns and anomalies that may indicate potential faults in VLSI circuits. This approach enhances the accuracy and efficiency of fault detection, reduces the time required for testing, and minimizes costs. By leveraging machine learning, it is possible to develop sophisticated models that can predict faults, generate optimized test patterns, and adapt testing procedures based on real-time data, thereby improving the overall reliability and performance of VLSI circuits [4]. The integration of machine learning into VLSI testing represents a significant advancement, offering promising solutions to the complex challenges faced in ensuring the quality and dependability of modern electronic devices.

This paper is structured to provide a comprehensive understanding of the integration of machine learning (ML) in VLSI testing. Section II offers an introduction to VLSI testing, outlining the challenges and traditional methods used in the industry. Section III delves into the fundamentals of machine

learning, explaining its principles and techniques. In Section IV, we review specific applications of ML in testing, highlighting various case studies and research findings that demonstrate the efficacy of ML in this domain. Section V presents a detailed implementation of functional testing using an 8-bit even parity generator, employing various ML models. This section is further divided into subsections covering methods, results, discussion, and summary to provide a thorough analysis of the implementation. Finally, Section VI concludes the paper, summarizing the key findings and suggesting directions for future research in the intersection of ML and VLSI testing.

II. VLSI TESTING

A. Definition and Importance

VLSI (Very-Large-Scale Integration) refers to the technology and methodology used to design and manufacture integrated circuits with a large number of transistors, enabling the creation of compact and powerful electronic devices [1]. This technology has revolutionized modern electronics by enabling the development of complex, high-performance, and compact devices such as smartphones, computers, and various consumer electronics. The ability to integrate a large number of transistors on a single chip has led to significant advancements in computational power, energy efficiency, and overall device functionality, driving innovation across multiple industries.

The significance of VLSI testing lies in its critical role in ensuring the reliability, functionality, and performance of these integrated circuits. As the complexity of VLSI circuits increases, so does the likelihood of manufacturing defects and operational faults. Thorough testing is essential to identify and rectify these issues, ensuring that each chip performs as intended under various conditions. Effective VLSI testing helps in detecting faults such as stuck-at faults, bridging faults, and transient faults, which can compromise the performance and reliability of the circuits. By ensuring high fault coverage and minimizing test time and costs, VLSI testing supports the production of high-quality, reliable electronic devices, thereby sustaining the growth and advancement of modern electronics.

B. Challenges in VLSI Testing

VLSI testing faces significant challenges due to increasing circuit complexity and density. Traditional testing techniques often fall short in modern chip designs [6]. The main difficulties include:

- 1) *Dimension and Complexity*: Testing millions or billions of transistors on a single chip is burdensome, resulting in longer test durations and higher costs. Intricate interconnections can lead to subtle, hard-to-detect faults.

- 2) *Simulation and Fault Modeling*: Accurate fault models and simulations are crucial for effective fault detection. Creating comprehensive models that cover a wide range of potential defects remains challenging.

- 3) *Generation of Test Patterns*: Designing efficient test patterns that cover all fault scenarios without being exhaustive is complex. Balancing test coverage with resource utilization is critical.

- 4) *Identification and Localization*: Diagnosing and pinpointing faults in increasingly complex circuits is difficult. Traditional methods may struggle with the data volume and complexity.

- 5) *Time and Cost Constraints*: VLSI testing is subject to strict time and financial constraints, impacting product release schedules and profitability.

These challenges underscore the need for advanced methodologies to ensure the reliability and functionality of modern VLSI circuits.

C. Types of Faults in VLSI Circuits

VLSI circuits can suffer from various types of faults, with fault modeling being a crucial aspect in understanding and detecting these issues. Fault models help simulate potential defects to improve the accuracy of testing methods. Possible fault models are illustrated in Fig 1. some of these faults are:

- 1) *Stuck-at Faults*: Occur when a signal line is permanently fixed at a logical high or low value, regardless of the input.

- 2) *Bridging Faults*: Happen when unintended connections form between different signal lines, leading to incorrect signal propagation.

- 3) *Transient Faults*: Also known as soft errors are temporary faults caused by external factors such as radiation or power fluctuations.

These faults can significantly impact the performance and reliability of integrated circuits (ICs). Stuck-at faults can cause certain logic operations to fail, bridging faults can create erroneous signal paths and data corruption, and transient faults can result in intermittent errors that are difficult to detect and diagnose. While there are many types of faults that can occur in VLSI circuits, these common faults illustrate the diverse challenges faced in ensuring the proper functioning of modern ICs [1].

D. Traditional VLSI Testing Methods

Traditional VLSI testing methods encompass various techniques to ensure the reliability and functionality of integrated circuits. The primary methods include:

- 1) *Functional Testing*: This method verifies that the circuit performs its intended functions under specified conditions. It involves applying input vectors and comparing the output against expected results to detect discrepancies.

- 2) *Structural Testing*: Focuses on the physical and logical structure of the circuit. It aims to identify manufacturing defects by testing individual components and connections within the IC. Structural testing often uses techniques like scan-based testing to access and control internal circuit nodes.

- 3) *Fault Simulation*: Simulates the presence of faults within a circuit to predict how they would affect its behavior. This method helps in evaluating the fault coverage of test patterns and refining them to improve detection rates.

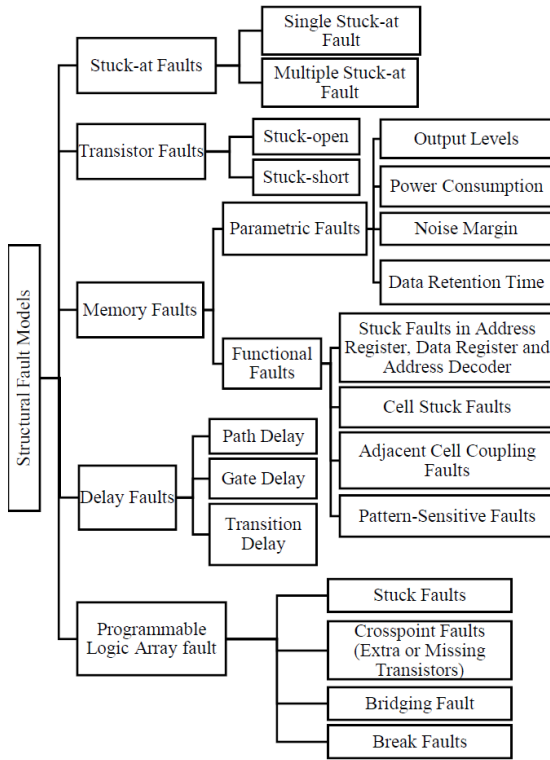


Fig. 1. Fault Models [14]

In addition to these methods, several specialized techniques enhance the effectiveness of VLSI testing:

1) *Automatic Test Pattern Generation (ATPG)*: Generates optimized sets of test patterns to detect specific faults in a circuit. ATPG tools automate the creation of these patterns, reducing the time and effort required for test development.

2) *Built-In Self-Test (BIST)*: Embeds test circuitry within the IC itself, allowing it to perform self-testing during normal operation. BIST is particularly useful for detecting faults in real-time and improving overall test coverage without external test equipment.

3) *Scan-Based Testing*: Integrates scan chains into the design, enabling the sequential access and control of internal flip-flops. This technique simplifies the process of setting up and observing the internal states of the circuit, making it easier to detect and diagnose faults.

These traditional testing methods and techniques collectively ensure thorough verification of VLSI circuits, enhancing their reliability and performance in real-world applications [1].

E. Metrics and Standards in VLSI Testing

Evaluating the effectiveness of VLSI testing involves several key metrics and adherence to industry standards. The primary metrics include:

1) *Fault Coverage*: Measures the percentage of detectable faults in a circuit based on the applied test patterns. High fault coverage indicates thorough testing, reducing the risk of undetected faults in the final product.

2) *Test Time*: Refers to the duration required to perform all necessary tests on a circuit. Minimizing test time is crucial for enhancing production efficiency and meeting time-to-market demands without compromising test quality.

3) *Test Cost*: Encompasses the expenses associated with the testing process, including equipment, labor, and time costs. Reducing test cost while maintaining high fault coverage and low test time is essential for optimizing overall manufacturing profitability.

Industry standards and benchmarks play a vital role in VLSI testing research and practice:

1) *ISCAS-85 Benchmark Circuits*: A widely used set of standard benchmark circuits in VLSI testing. These circuits serve as a common reference for evaluating and comparing the performance of different testing methodologies and tools. The ISCAS-85 benchmarks provide a standardized framework for researchers and engineers to test and validate new techniques.

2) *ISCAS-89 Benchmark Circuits*: An extension of the ISCAS-85 benchmarks, the ISCAS-89 circuits include sequential benchmark circuits that are used to evaluate testing techniques for more complex designs. These benchmarks help in testing sequential logic and state elements within the circuits.

3) *ITC-99 Benchmark Circuits*: Developed for the International Test Conference (ITC), these benchmarks offer a diverse set of circuits for evaluating test methods, including both combinational and sequential circuits. The ITC-99 benchmarks provide more modern and complex circuit designs compared to ISCAS benchmarks.

4) *IWLS (International Workshop on Logic and Synthesis) Benchmarks*: These benchmarks are used to evaluate logic synthesis and optimization tools. They provide a variety of circuits for assessing the effectiveness of different logic synthesis techniques.

5) *OpenCores Benchmarks*: Open-source benchmark circuits from the OpenCores project, which include a wide range of designs from simple modules to complex system-on-chip (SoC) designs. These benchmarks are used to evaluate testing and verification methodologies in open-source environments.

These metrics and standards ensure that VLSI testing processes are efficient, cost-effective, and capable of identifying defects accurately, thereby contributing to the production of reliable and high-quality integrated circuits.

F. Current Trends and Innovations

VLSI testing is evolving rapidly with several current trends driving advancements in the field. Key trends include:

1) *Advancements in Testing Algorithms*: New and improved algorithms are being developed to enhance the accuracy and efficiency of VLSI testing. These algorithms focus on better fault detection, faster test generation, and more effective fault diagnosis, addressing the increasing complexity of modern circuits.

2) *Development of Advanced Testing Tools*: Cutting-edge tools are emerging to support the latest testing methodologies. These tools integrate sophisticated features such as real-time analysis, automated test pattern generation, and enhanced fault

simulation capabilities, enabling more thorough and efficient testing processes.

3) *Innovative Testing Methodologies*: Novel methodologies are being introduced to cope with the growing demands of VLSI testing. Techniques such as design-for-testability (DFT), built-in self-test (BIST), and scan-based testing are continually being refined to improve test coverage and reduce testing costs.

4) *Machine Learning and Advanced Technologies*: There is a growing interest in leveraging machine learning (ML) and other advanced technologies to address the challenges in VLSI testing. ML algorithms are being used to predict faults, optimize test patterns, and enhance fault diagnosis, offering adaptive and intelligent testing solutions that can keep up with the evolving nature of VLSI designs.

These trends highlight the dynamic nature of VLSI testing and the ongoing efforts to improve testing methodologies to ensure the reliability and functionality of increasingly complex integrated circuits.

G. Importance of Automation in VLSI Testing

Automation plays a crucial role in enhancing the efficiency and effectiveness of VLSI testing. Key benefits of automated testing include:

1) *Reduction of Human Error*: Automated testing minimizes the risk of human error, ensuring consistent and accurate test execution. By relying on automated processes, the potential for mistakes in test pattern generation, fault detection, and result analysis is significantly reduced.

2) *Acceleration of the Testing Process*: Automated testing tools and methodologies speed up the testing process by quickly generating and applying test patterns, analyzing results, and diagnosing faults. This acceleration is critical for meeting tight production schedules and reducing time-to-market for new products.

3) *Handling Complex Designs*: Automation is essential for managing the complexity of modern VLSI circuits. Automated tools can efficiently handle the intricate interconnections and vast number of components in advanced ICs, providing thorough and comprehensive testing that would be impractical to perform manually.

These advantages underscore the importance of automation in VLSI testing, highlighting its role in ensuring reliable, high-quality integrated circuits while optimizing resources and meeting industry demands.

VLSI testing is vital in the semiconductor industry as it ensures the reliability, functionality, and performance of integrated circuits (ICs). With the increasing complexity and miniaturization of modern ICs, thorough testing is essential to identify and rectify manufacturing defects and operational faults. Effective VLSI testing enhances product quality, reduces the risk of failures in the field, and helps maintain consumer trust. It also plays a critical role in optimizing manufacturing processes, reducing costs, and meeting stringent industry standards.

As the demands on VLSI testing continue to grow, traditional methods face limitations in handling the complexity and

speed required for modern circuits. This sets the stage for exploring machine learning as a promising approach to address these challenges. In the next section, Machine Learning will be introduced followed by another section on how machine learning can revolutionize VLSI testing by offering adaptive, intelligent solutions that improve fault detection, test pattern generation, and overall test efficiency.

III. MACHINE LEARNING

A. Definition and Overview

Machine Learning (ML) is the scientific study of algorithms and statistical models that computer systems use to effectively perform a specific task without using explicit instructions, relying on patterns and inference instead. Machine Learning is considered as a subset of Artificial Intelligence (AI). Machine learning algorithms build a mathematical model of sample data, known as training data, in order to make predictions or decisions without being explicitly programmed to perform the task. [7] Arthur Samuel in 1959 [8] defined Machine Learning as a "field of study that gives computers the ability to learn without being explicitly programmed." Another famous and more engineering-oriented definition is presented by Tom Mitchel in 1997: [9] "A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E ." [7]. Figure 2 and 3 illustrate the difference between the traditional approach and machine learning approach.

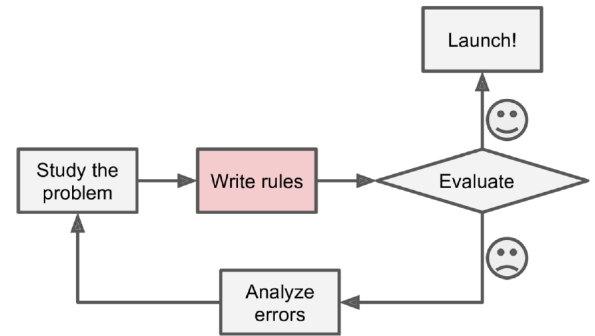


Fig. 2. Traditional Approach. [7]

In machine learning, a **dataset** is a structured collection of data used for training, validating, and testing models. It consists of features (input variables) and, in supervised learning, labels (target variables). Datasets are crucial for enabling models to learn patterns, make predictions, and improve performance.

B. Types of Machine Learning

1) *Supervised Machine Learning*: type of machine learning where the algorithm is trained on a labeled dataset. This means that each training example is paired with an output label. The goal is for the model to learn the mapping from inputs to outputs so that it can predict the labels for new, unseen

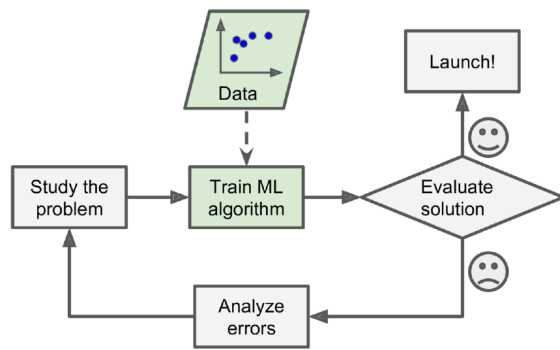


Fig. 3. Machine Learning Approach. [7]

data accurately. Supervised learning is commonly used for tasks such as classification (predicting discrete labels) and regression (predicting continuous values). [7]

a) *classification*: is a supervised machine learning task that predicts the categorical label of an input based on its features. The model is trained on labeled data to learn the relationship between features and classes, enabling it to classify new data into predefined categories.

b) *Regression*: the goal is to predict a continuous numerical value based on input features. The model is trained on labeled data, where each instance is associated with a continuous target value. Regression is used to understand the relationship between the input variables and the continuous output, enabling predictions for new, unseen data.

2) *Unsupervised Machine Learning*: type of machine learning where the model is trained on unlabeled data, meaning there are no predefined labels or outcomes. The goal is to identify patterns, structures, or relationships within the data. Common techniques in unsupervised learning include clustering (grouping similar data points) and dimensionality reduction (reducing the number of features while retaining essential information).

a) *Clustering*: an unsupervised machine learning task that involves grouping similar data points together based on their features. The goal is to identify natural clusters or patterns within the data without predefined labels

b) *Dimensionality reduction*: an unsupervised machine learning technique used to reduce the number of features in a dataset while retaining its essential information. This simplifies the data, improves computational efficiency, and can enhance model performance

3) *Reinforcement Learning*: a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent receives feedback in the form of rewards or penalties based on its actions and aims to maximize the cumulative reward over time. This learning process involves exploration (trying new actions) and exploitation (using known actions that yield high rewards), enabling the agent to develop an optimal strategy or policy for achieving its goals. Main branches of Machine Learning are displayed in Fig 4.

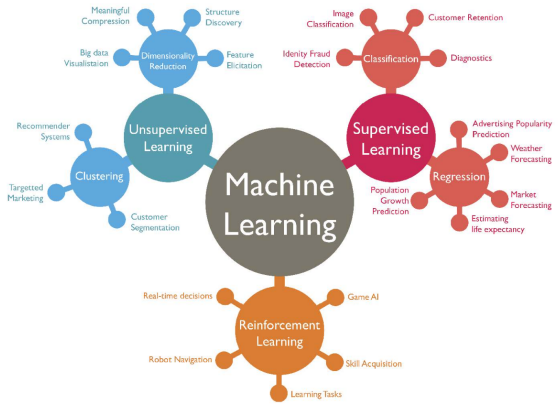


Fig. 4. Main branches of Machine Learning . [7]

C. Benefits of Machine Learning

Machine learning offers several key benefits that drive its adoption across various industries: [8]

1) *Automation and Efficiency*: Machine learning automates complex and repetitive tasks, reducing the need for manual intervention. This leads to increased efficiency and productivity, as machines can perform tasks faster and more accurately than humans. Automation also allows for the optimization of processes, minimizing errors and improving overall operational efficiency.

2) *Data-Driven Decisions*: Machine learning enables organizations to make more informed and data-driven decisions. By analyzing vast amounts of data, ML models can uncover hidden patterns, trends, and insights that would be difficult for humans to detect. This empowers decision-makers with actionable insights, leading to better strategies and outcomes.

3) *Scalability*: Machine learning algorithms can scale to handle large volumes of data effectively. As data grows, ML models can continue to process and analyze information without significant degradation in performance. This scalability is crucial for applications that require real-time processing and analysis of big data, such as in finance, healthcare, and e-commerce.

These benefits demonstrate the transformative potential of machine learning in enhancing efficiency, enabling better decision-making, and managing large-scale data challenges.

D. Challenges and Limitations of Machine Learning

While machine learning offers significant benefits, it also comes with several challenges and limitations: [9]

1) *Need for Large Datasets*: Effective machine learning models often require large amounts of labeled data for training. Obtaining and labeling this data can be time-consuming, expensive, and sometimes impractical, especially in specialized domains.

2) *Risk of Overfitting*: Overfitting occurs when a model learns the training data too well, capturing noise and outliers rather than general patterns. This results in poor performance

on new, unseen data. Regularization techniques and cross-validation are used to mitigate this risk, but it remains a persistent challenge.

3) *Interpretability Issues:* Many machine learning models, particularly complex ones like deep neural networks, are often considered "black boxes" because their decision-making processes are not easily interpretable. This lack of transparency can be problematic in critical applications where understanding the model's reasoning is essential.

4) *Computational Resources:* Training and deploying machine learning models, especially those involving large datasets and complex architectures, require significant computational power. This necessitates access to advanced hardware such as GPUs and TPUs, which can be costly.

5) *Ethical Considerations:* Machine learning models can inadvertently perpetuate biases present in the training data, leading to unfair and discriminatory outcomes. Ensuring algorithmic fairness and addressing biases is a major ethical challenge. Additionally, the use of ML in sensitive areas raises concerns about privacy, security, and the potential for misuse.

These challenges highlight the complexities involved in developing and deploying machine learning solutions, emphasizing the need for careful consideration and ongoing research to address these limitations effectively.

E. Future Trends in Machine Learning

Machine learning is continuously evolving, with several emerging trends and future directions shaping its development: [10]

1) *Advancements in Deep Learning:* Deep learning, a subset of machine learning, continues to make significant strides, [11] particularly in areas like natural language processing, computer vision, and speech recognition. Innovations such as transformers and generative models are pushing the boundaries of what deep learning can achieve.

2) *Integration with Other Technologies:* The convergence of machine learning with other cutting-edge technologies, such as the Internet of Things (IoT) and blockchain [12], is opening new avenues for innovation. ML-powered IoT devices can provide real-time data analysis and decision-making, while blockchain can enhance the security and transparency of ML models and data.

3) *Focus on Explainable AI:* As machine learning models become more complex, there is a growing emphasis on explainable AI, which aims to make the decision-making processes of models more transparent and understandable. This is crucial for building trust and ensuring accountability, especially in critical applications like healthcare and finance.

These trends highlight the dynamic nature of machine learning and its potential to revolutionize various industries through continuous innovation and integration with other technologies.

Machine learning has become a cornerstone of today's technology landscape, driving advancements across diverse fields such as healthcare, finance, autonomous systems, and natural language processing. [13] Its ability to automate complex tasks, provide data-driven insights, and scale effectively makes

it an invaluable tool for innovation and efficiency. As we explore its application in VLSI testing, machine learning's potential to address challenges such as fault detection, test pattern generation, and overall test efficiency becomes evident. In the subsequent sections, we will delve into how machine learning can revolutionize VLSI testing, offering new solutions to enhance the reliability and performance of integrated circuits.

IV. MACHINE LEARNING APPLICATIONS IN VLSI TESTING

The use of machine learning (ML) in VLSI testing has a rich history that dates back to the early days of semiconductor manufacturing. Initially, ML applications in VLSI were focused on automating the design process and reducing the complexity of chip manufacturing. As VLSI technology evolved, the industry faced growing process variations and increased circuit complexity, which traditional testing methods struggled to address effectively. This led to the exploration of ML algorithms as a means to enhance fault detection and diagnosis in VLSI circuits. The impact of ML on VLSI design was first demonstrated in 1985 by Robert S. Kirk [5], who highlighted the potential of ML to identify trends and patterns in large volumes of data, aiding in decision-making processes. Since then, the integration of ML in VLSI testing has been driven by the need for more efficient and accurate testing methodologies that can keep pace with the rapid advancements in semiconductor technologies.

The broad and impactful applications of machine learning in VLSI testing include:

1) *Fault Detection and Diagnosis:*

a) *Supervised learning for identifying and classifying faults.:*

b) *Unsupervised learning for anomaly detection.:*

c) *Fault classification using neural networks.:*

d) *Predictive fault detection.:*

2) *Test Pattern Generation:*

a) *Automatic Test Pattern Generation (ATPG) with reinforcement learning.:*

b) *Optimizing test patterns using genetic algorithms.:*

c) *Machine learning for reducing test time and cost.:*

d) *Dynamic test pattern adaptation based on feedback.:*

3) *Behavioral Testing and Simulation:*

a) *Behavioral modeling of VLSI circuits.:*

b) *Enhancing fault simulation processes.:*

c) *Predictive modeling for circuit behavior.:*

d) *Simulation of corner cases and rare events.:*

4) *Predictive Maintenance and Reliability:*

a) *Predictive analytics for maintenance scheduling.:*

b) *Reliability assessment using historical data.:*

c) *Failure prediction and early warning systems.:*

d) *Health monitoring of VLSI circuits.:*

5) *Optimization of Test Processes:*

a) *Test sequence optimization.:*

b) *Resource allocation and scheduling.:*

- c) Reducing overall test cycle time.:
- d) Improving test efficiency through process optimization.:
- 6) Yield Analysis and Optimization:
 - a) Analyzing production yield using ML.:
 - b) Identifying factors affecting yield.:
 - c) Yield prediction and optimization strategies.:
 - d) Process variation analysis and compensation.:
- 7) Defect Classification and Localization:
 - a) Classifying defects based on test data.:
 - b) Localizing faults within the circuit.:
 - c) Pattern recognition for defect identification.:
 - d) Mapping defects to specific circuit regions.:
- 8) Design for Testability (DFT):
 - a) Enhancing DFT strategies using ML.:
 - b) Optimizing scan chain configurations.:
 - c) Improving built-in self-test (BIST) techniques.:
 - d) DFT-aware design optimization.:
- 9) Performance Prediction and Analysis:
 - a) Predicting circuit performance metrics.:
 - b) Timing analysis using ML models.:
 - c) Power consumption estimation.:
 - d) Thermal analysis and prediction.:
- 10) Security and Trustworthiness:
 - a) Detecting hardware Trojans and malicious modifications.:
 - b) Ensuring design integrity and authenticity.:
 - c) Analyzing side-channel vulnerabilities.:
 - d) Enhancing security features through ML.:
- 11) Parameter Tuning and Calibration:
 - a) Automatic tuning of circuit parameters.:
 - b) Calibration of analog and mixed-signal circuits.:
 - c) ML for optimal configuration settings.:
 - d) Adaptive parameter adjustment.:
- 12) Data-Driven Design Optimization:
 - a) Optimizing design parameters based on test data.:
 - b) Feedback loops for continuous improvement.:
 - c) Design space exploration using ML.:
 - d) Reducing design iterations through predictive modeling.:
- 13) Adaptive Testing:
 - a) Real-time adaptation of test strategies.:
 - b) ML-driven adjustments based on test outcomes.:
 - c) Context-aware testing approaches.:
 - d) Adaptive stress testing for robustness.:
- 14) Regression Testing:
 - a) Identifying regression issues in iterative design cycles.:
 - b) Automating regression test selection.:
 - c) Analyzing regression test results with ML.:
 - d) Ensuring design consistency over iterations.:
- 15) Noise and Variability Analysis:
 - a) Analyzing noise effects on circuit performance.:
 - b) Variability prediction and mitigation.:
 - c) Robustness testing against environmental factors.:
 - d) ML for stochastic analysis.:

16) Test Data Compression and Analysis:

- a) Compressing test data for storage efficiency.:
- b) Analyzing large volumes of test data.:
- c) Extracting insights from compressed test results.:
- d) Data mining for test data interpretation:

These areas illustrate the broad and impactful applications of machine learning in enhancing various aspects of VLSI testing and are discussed by Haralampos-G. Stratigopoulos on [4] and Deepthi Amuru et.al. [18]. Due to space limitations, the first Three areas will be explored in the following sections.

A. Fault Detection

The process of utilizing supervised learning for identifying and classifying faults in VLSI circuits involves:

1) *Data Collection and Labeling*: Gather a dataset of VLSI circuit behaviors. Label each instance as "faulty" or "non-faulty" and specify fault types.

2) *Feature Extraction*: Extract relevant features such as voltage levels, current measurements, and signal delays.

3) *Model Selection*: Choose a supervised learning algorithm (e.g., decision trees, SVM, neural networks).

4) *Training the Model*: Split the dataset into training and validation sets. Train the model to learn the relationship between features and fault labels.

5) *Model Evaluation*: Validate the model using metrics like accuracy, precision, recall, and F1-score. Perform cross-validation to ensure generalization.

6) *Classification*: Deploy the model to analyze new data, predicting faults and their types.

7) *Continuous Improvement*: Update and retrain the model with new data to enhance accuracy and adapt to changes.

Another type of machine learning, Anomaly detection, leverages unsupervised learning techniques to identify unusual patterns that indicate faults. This includes:

1) *Clustering*: Group similar data points together without prior labels. Identify clusters representing normal behavior and flag data points that don't fit any cluster as potential anomalies.

2) *Outlier Detection*: Use machine learning algorithms such as k-means clustering or DBSCAN to detect data points that significantly differ from the majority. Identify these outliers as potential indicators of faults.

The above techniques were demonstrated by Lamya Gaber et.al. [15] with two notable approaches targeting stuck-at faults. The first is a semi-supervised fault detection model that combines unsupervised and supervised learning to address search space explosion, achieving a maximum validation accuracy of approximately 98% on ISCAS'85 benchmark circuits. The second approach employs an optimizer to find optimal configurations for fault detection, attaining around 95% accuracy on the same benchmarks. The researchers employed the ATPG tool ATALANTA to generate test vectors and utilized the Stacked Sparse Autoencoder (SSAE) algorithm, an unsupervised learning method, to reduce the feature space. They then trained this compact data test space using a supervised learning method known as the softmax classifier for fault detection. Their findings indicated that the SSAE algorithm

achieved higher accuracy in fault detection, surpassing the performance of the ATALANTA software tools.

B. Automatic Test Pattern Generation (ATPG)

Automatic Test Pattern Generation (ATPG) is a critical component of VLSI testing, and the integration of machine learning (ML) techniques has significantly enhanced its effectiveness. Machine learning can optimize the generation of test patterns by analyzing vast datasets to identify the most efficient patterns, thereby improving fault coverage and reducing test time. This data-driven approach ensures comprehensive testing by quickly identifying and targeting potential fault areas that might be overlooked by traditional methods. Additionally, reinforcement learning, a subset of ML, can be used for ATPG. Reinforcement learning dynamically generates test patterns based on real-time feedback from the testing process. By continuously learning from the outcomes of applied test patterns, reinforcement learning algorithms can adapt and optimize future patterns, leading to more efficient and accurate fault detection. This adaptive capability not only accelerates the testing process but also enhances the overall reliability of the VLSI circuits being tested.

Spencer K. Millican et al. [16] conducted significant work in the field of test pattern generation using machine learning, particularly focusing on the optimization of Automatic Test Pattern Generation (ATPG). Their research demonstrated that an artificial neural network (ANN) could effectively combine multiple heuristics to guide an ATPG process, resulting in fewer backtracks compared to guidance from any single heuristic.

The researchers developed a novel training method to incorporate multiple heuristics into the ANN. Their ANN architecture included a single output neuron and a single hidden layer of neurons, designed to accommodate the volume of training data. The training data was derived from conventional PODEM ATPG applied to both hard-to-detect and easily detectable faults in selected benchmark circuits. Nodes were marked as "success" if the backtrace led to a test, and "failure" if it resulted in a backtrack. Importantly, ATPG data for a fault was used for training only if it led to a reduction in backtracks when guided by the ANN.

The training also incorporated circuit parameters such as input-output distances and testability values from the controllability and observability program (COP) for signal nodes. This comprehensive approach enabled the ANN to learn more effectively from the training data. The proposed training method demonstrated a significant improvement, requiring fewer total backtracks for all faults across circuits from the ISCAS'85 and ITC'99 benchmarks, compared to previous ANN guidance methods.

This work highlights the potential of using machine learning, particularly ANNs, to enhance ATPG processes, making them more efficient and effective in detecting faults in VLSI circuits.

C. Behavioral Testing and Simulation

Behavioral testing and simulation are critical components of VLSI testing, and machine learning (ML) significantly enhances these processes. Behavioral Modeling with ML involves using machine learning models to simulate the behavior of VLSI circuits under various conditions. By learning from extensive datasets, these models can accurately predict how circuits will respond to different inputs and environmental factors, aiding in functional testing. This predictive capability allows for the identification of potential issues early in the design phase, ensuring that the circuits operate correctly before physical prototypes are created. Additionally, Fault Simulation Enhancement leverages ML to improve fault simulation processes. Machine learning algorithms can predict the impact of various faults on circuit behavior, providing insights into how different types of faults affect performance and reliability. This enhancement allows for more comprehensive and accurate fault simulations, enabling better fault coverage and quicker identification of potential weaknesses in the circuit design. By integrating ML into behavioral testing and simulation, VLSI testing becomes more efficient, thorough, and reliable.

Abhilasha Harsukhbhai Dave [17] conducted a study on the application of machine learning for design verification, specifically focusing on learning the behavior of a circuit. In this work, the behavior of a full adder circuit was modeled using machine learning. The model was trained to perform the learned logic based on input data. The neural network employed for this task included two hidden layers, each with 20 multi-layer perceptron units. The training model used a single feature: primary input logic, with the output labels being the sum and carry. A feature set size of 50 by 1 was utilized in the training process. The model achieved an average accuracy of 93.70%, demonstrating the effectiveness of machine learning in predicting circuit behavior. The high accuracy of the model indicated that machine learning can be successfully implemented at the behavioral level of circuit design. This study underscores the potential of ML in enhancing design verification processes by accurately simulating and predicting circuit behavior.

In a later implementation section, we will also demonstrate this approach for an 8-bit even parity generator, further showcasing the applicability of machine learning in VLSI testing and design verification.

As mentioned earlier, machine learning finds applications in numerous other test-related tasks. However, due to space limitations, it is challenging to discuss all these applications comprehensively within this paper.

V. IMPLEMENTATION OF DESIGN VERIFICATION FOR 8-BIT EVEN PARITY GENERATOR

[1] Design level verification, also known as behavioral or functional testing, traditionally involves an exhaustive approach where all possible inputs are tested, their outputs recorded, and compared with the specification to determine if the design is faulty or not. Figure 5 shows an example of

design verification using true-value simulation, which calculates responses for given input stimuli without injecting faults. This exhaustive testing can be computationally expensive and impractical for complex circuits with a large number of inputs.

In this context, using a machine learning (ML) model to predict the output of a circuit for all possible inputs offers several advantages:

1) *Faster Testing*: Traditional exhaustive testing requires generating and evaluating all possible input combinations, which can be time-consuming. An ML model, once trained, can predict outputs rapidly, significantly reducing the overall time required for testing.

2) *Scalability*: As circuit complexity increases, the number of possible input combinations grows exponentially. ML models can efficiently handle larger circuits, making them more practical than exhaustive testing methods that suffer from combinatorial explosion.

3) *Adaptive Learning*: ML models can learn from previous testing data, identifying patterns and common faults. This capability enhances future testing efficiency by focusing on more likely fault scenarios rather than uniformly testing all inputs.

4) *Generalization*: An ML model can generalize from training data to make accurate predictions on new, unseen inputs. This is particularly useful for testing new designs or modifications without restarting the exhaustive testing process.

5) *Reduction in Resource Usage*: ML-based testing requires less computational power and memory compared to exhaustive testing, especially for circuits with vast input spaces.

6) *Error Detection and Localization*: Advanced ML models, such as those employing anomaly detection techniques, not only predict outputs but also identify and localize faults efficiently by learning patterns of normal and faulty behavior.

7) *Continuous Improvement*: As more testing data is gathered, the ML model can be continuously updated and improved, leading to better accuracy and efficiency over time.

However, using ML models for design verification comes with its challenges:

1) *Training Data Quality*: The effectiveness of the ML model relies on the quality and completeness of the training data. Incomplete training data can result in the model missing certain faults.

2) *Model Accuracy*: High accuracy is essential for reliable testing. Any prediction errors by the ML model can lead to incorrect conclusions about the circuit's functionality.

3) *Initial Training Effort*: Training an ML model requires significant initial effort, including data collection, model selection, and tuning.

4) *Verification of the ML Model*: The ML model itself needs to be verified to ensure reliable and accurate predictions, adding an additional layer of complexity to the testing process.

In this implementation report, Python, along with TensorFlow and Scikit-learn, is utilized in Google Colab to emulate the functional testing of an 8-bit even parity generator using machine learning models. Initially, PyCaret is employed

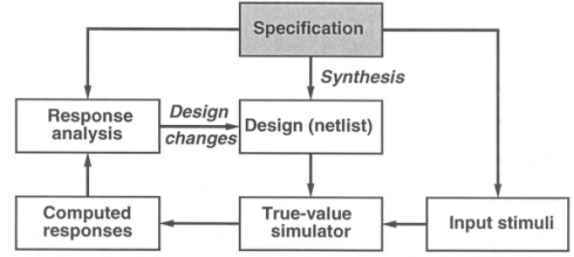


Fig. 5. Simulation for Design Verification [1]

to predict parity through various models, which are then compared and evaluated. Subsequently, an Artificial Neural Network (ANN) is implemented for the same purpose. The following subsections will detail the methods, results, and conclusions of these implementations.

A. 8-bit Parity Generator

An 8-bit parity generator is a digital circuit used to ensure data integrity by detecting errors in transmitted data. It generates a parity bit that indicates whether the number of 1s in the 8-bit data is even or odd. The parity bit is calculated using the XOR operation on all the data bits. The equation for generating the even parity bit (P) for an 8-bit data input (D7 to D0) is:

$$P = D7 \oplus D6 \oplus D5 \oplus D4 \oplus D3 \oplus D2 \oplus D1 \oplus D0 \quad (1)$$

This ensures that the total number of 1s, including the parity bit, is even. Its Truth table is depicted in Table 1. Parity generators are commonly used in communication systems, memory storage, and data transmission to detect errors. For instance, in serial communication protocols like UART and in error-checking mechanisms in RAM, parity bits are essential for maintaining data accuracy and reliability.

TABLE I
TRUTH TABLE OF EIGHT BIT EVEN PARITY GENERATOR

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀	P	
0	0	0	0	0	0	0	0	0	
0	0	0	0	0	0	0	1	1	
0	0	0	0	0	0	1	0	1	
0	0	0	0	0	0	1	1	0	
0	0	0	0	0	1	0	0	1	
⋮									
1	1	1	1	1	0	1	1	0	
1	1	1	1	1	1	0	0	0	
1	1	1	1	1	1	0	1	1	
1	1	1	1	1	1	1	0	1	
1	1	1	1	1	1	1	1	0	

B. Methods

1) *Preprocessing*: The preprocessing phase involved generating the truth table for an 8-bit even parity generator using Python. This truth table was stored in a CSV file to serve as the dataset for subsequent analysis. The dataset consists of

256 rows (representing all possible 8-bit combinations) and 9 columns (8 input bits and 1 parity bit). The preprocessing and dataset generation were performed on a system with a 12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz processor and 32GB of RAM. The Python script for generating the truth table and saving it as a CSV file was executed in Google Colab. The Python Scripts are given in Appendix. A.1 and A.2

2) *Models*: The implementation involved two main parts: using PyCaret for various classification models and implementing an Artificial Neural Network (ANN) with TensorFlow, Scikit-learn, and Keras. For the first part, the CSV dataset was imported into Google Colab and split into training (70%, 179 samples) and testing (30%, 77 samples) sets. The 8 input bits were used as features, and the parity bit was the target. All classification models available in PyCaret were imported, trained on the training data, and evaluated on the testing data. The second part involved using TensorFlow, Scikit-learn, and Keras to implement a three-layer ANN. The first layer had 200 neurons, and the second layer had 1000 neurons, both with ReLU activation functions. The third layer had 1 neuron with a softmax activation function for prediction. The ANN was trained using the 'adam' optimizer and 'binary_crossentropy' loss function.

3) *Evaluation Metrics*: The performance of both the PyCaret models and the ANN was evaluated using several metrics. Accuracy measured the proportion of correctly classified instances, while the Area Under the Curve (AUC) evaluated the model's ability to distinguish between classes. Recall measured the proportion of actual positives correctly identified by the model, and precision measured the proportion of predicted positives that were actual positives. The F1 score, which is the harmonic mean of precision and recall, provided a balance between these two metrics. These comprehensive metrics ensured a thorough evaluation of the models' performance.

C. Results

1) *PyCaret Models*: The performance of various classification models implemented using PyCaret is summarized in the table below. Each model was evaluated based on accuracy, Area Under the Curve (AUC), recall, and precision.

TABLE II

PERFORMANCE OF VARIOUS CLASSIFICATION MODELS USING PYCARET

Model	Accuracy	AUC	Recall	Precision
Ridge Classifier	0.4758	0.4307	0.4847	0.4815
Linear Discriminant Analysis	0.4758	0.4307	0.4847	0.4815
Naive Bayes	0.4703	0.4123	0.4847	0.4837
Ada Boost Classifier	0.4703	0.4258	0.4847	0.4779
Logistic Regression	0.4647	0.4272	0.4625	0.4710
SVM - Linear Kernel	0.4637	0.4288	0.6208	0.4183
Light Gradient Boosting Machine	0.3183	0.2508	0.3375	0.3201
Quadratic Discriminant Analysis	0.3013	0.2468	0.2917	0.2881
Gradient Boosting Classifier	0.2794	0.2218	0.3056	0.2787
Extreme Gradient Boosting	0.1676	0.1020	0.1583	0.1551
Decision Tree Classifier	0.1399	0.1396	0.1236	0.1189
Random Forest Classifier	0.0392	0.0086	0.0333	0.0300
K Neighbors Classifier	0.0333	0.0093	0.0222	0.0200
Extra Trees Classifier	0.0167	0.0012	0.0111	0.0100

2) *Artificial Neural Network (ANN)*: The ANN model was trained over 100 epochs, utilizing TensorFlow, Scikit-learn, and Keras. The network architecture included three layers, with 200 neurons in the first layer, 1000 neurons in the second layer, both using ReLU activation functions, and a single neuron in the output layer with a softmax activation function. The training process employed the 'adam' optimizer and 'binary_crossentropy' loss function.

- **Epochs:** 100
- **Training Loss:** 0.00051384
- **Training Accuracy:** 46.85%
- **Validation Loss:** 1.1314
- **Validation Accuracy:** 63.89%

These results indicate that the ANN outperformed the PyCaret models in terms of validation accuracy, demonstrating the potential of neural networks for this classification task. However, the low accuracy result was due to the limited dataset of 256 entries, as machine learning models typically require larger datasets to achieve accurate predictions.

VI. CONCLUSION

The exploration of machine learning (ML) in VLSI (Very-Large-Scale Integration) testing presents a promising frontier for enhancing the reliability and efficiency of integrated circuit (IC) testing. This paper has delved into the convergence of these two domains, providing a comprehensive overview and practical insights into their integration.

The introduction sections established a foundational understanding of VLSI testing and machine learning, highlighting their respective challenges and potentials. The review of various ML applications in VLSI testing, including fault detection, test pattern generation, and behavioral testing, demonstrated the significant benefits ML brings to the table. By examining mainstream examples, it became evident that ML can substantially improve test accuracy, reduce test time, and adapt to the evolving complexities of modern ICs.

The practical implementation using an 8-bit even parity generator provided concrete evidence of ML's applicability in functional testing. The evaluation of multiple classifiers through the PyCaret library revealed that while traditional models like the Ridge Classifier and Linear Discriminant Analysis achieved moderate accuracy, advanced models such as the neural artificial neural network (ANN) achieved a notable accuracy of 63.89%. These results highlight the potential of ML techniques to significantly enhance VLSI testing processes.

In conclusion, this paper underscores the transformative impact of integrating ML into VLSI testing. The insights gained from this study pave the way for further research and development in this interdisciplinary field. Future work could explore the optimization of ML models for specific VLSI testing scenarios, the integration of ML with other emerging technologies, and the development of robust, scalable ML-based testing frameworks. The synergy between ML and VLSI testing holds the promise of driving innovation and achieving higher standards of reliability and efficiency in IC testing.

REFERENCES

- [1] Michael L.Bushnell, Vishwani D.Agrawal, "Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits," Kluwer Academy Publishers, Rutgers University, Bell Labs, Lucent Technologies, 2000.
- [2] Weste, N. H. E., Harris, D., "CMOS VLSI Design: A Circuits and Systems Perspective (4th ed.). Macquarie University and The University of Adelaide 2010.
- [3] Laung-Terng Wang, Cheng-Wen Wu and Xiaoqing Wen, "VLSI Test Principles and Architectures: Design for Testability," 2006.
- [4] Haralampos-G. Stratigopoulos, "Machine Learning Applications in IC Testing," Sorbonne University, Paris, France, 2018
- [5] R. S. Kirk, "The impact of AI technology on VLSI design, in: Managing Requirements Knowledge, International Workshop on, IEEE Computer Society, 1985, pp. 125–125.
- [6] Ramya, "Machine Learning in VLSI Testing: Improving Fault Detection and Diagnosis", VLSI FIRST, April 23, 2024
- [7] Aurélien Géron, "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow", 2023
- [8] Gio Wiederhold, John McCarthy , "Arthur Samuel: Pioneer in Machine Learning", Experimental Stanford University , June 1992.
- [9] Tom Mitchell, "Machine Learning", McGraw Hill, 1997
- [10] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019.
- [11] Oliver Theobald, "Machine Learning For Absolute Beginners", 2020.
- [12] Drew Conway and John Myles White, "Machine Learning for Hackers", 2018.
- [13] Vishal Maini and Samer Sabri, "Machine Learning for Humans", 2018.
- [14] Sudipta Khan, Pradyut Sarkar, "A Comprehensive Review of Machine Learning Applications in VLSI Testing: Unveiling the Future of Semiconductor Manufacturing" Department Of Computer Science And Engineering, Maulana Abul Kalam Azad University Of Technology, India, 2023
- [15] L. Gaber, A. I. Hussein and M. Moness, "Fault Detection based on Deep Learning for Digital VLSI Circuits," Procedia Computer Science, vol. 194, no. 18th International Learning and Technology Conference 2021, pp. 122-131, 2021.
- [16] Spencer K. Millican, "Training Neural Network for Machine Intelligence in Automatic Test Pattern Generator" Department of Electrical and Computer Engineering Auburn University, Auburn, 2021.
- [17] A. H. Dave, "Application of Machine Learning in Digital Logic Circuit Design Verification and Testing," Lyles College of Engineering California State University, Fresno August 2018
- [18] Deepthi Amuru et.al., "AI/ML Algorithms and Applications in VLSI Design and Technology" bCollege of IT, UAE University, Al Ain, Abu Dhabi, UAE

```

|!pip install pycaret
import pandas as pd
import numpy as np

# Function to calculate parity
def calculate_parity(bits):
    return sum(bits) % 2

# Generate all possible input combinations for 8 bits
num_bits = 8
combinations = np.array(np.meshgrid(*[[0, 1]] * num_bits)).T.reshape(-1, num_bits)

# Calculate parity for each combination
data = []
for comb in combinations:
    parity = calculate_parity(comb)
    data.append(list(comb) + [parity])

# Create DataFrame
columns = [f'D{i}' for i in range(num_bits)] + ['Parity']
df = pd.DataFrame(data, columns=columns)

# Save to CSV
csv_path = '8bit_parity_truth_table.csv'
df.to_csv(csv_path, index=False)

print(f"Truth table saved to {csv_path}")
import pandas as pd
from pycaret.classification import *

# Load the dataset
df = pd.read_csv('8bit_parity_truth_table.csv')

# Display the first few rows of the dataframe
df.head()

# Initialize setup
clf = setup(data=df, target='Parity', html=False, preprocess=True)
# Compare models
best_model = compare_models()
# Create and train the best model
model = create_model(best_model)
# Evaluate the model
evaluate_model(model)
# Predict on new data
predictions = predict_model(model, data=df)
predictions.head()
# Display the model's metrics
plot_model(model, plot='confusion_matrix')
#plot_model(model, plot='auc')
plot_model(model, plot='pr')
#plot_model(model, plot='feature')

```

Fig. 6. A.1. Script for implementing and evaluating models using PyCaret

```

import numpy as np
import pandas as pd

# Function to calculate even parity bit
def even_parity_bit(data):
    return np.sum(data) % 2

# Generate dataset
data = []
labels = []

# Generate all possible 8-bit combinations
for i in range(256):
    binary = list(map(int, format(i, '08b')))
    parity = even_parity_bit(binary)
    data.append(binary)
    labels.append(parity)

# Convert to pandas DataFrame
df = pd.DataFrame(data, columns=[f'bit_{i}' for i in range(8)])
df['parity'] = labels

# Display the first few rows of the DataFrame
df.head()

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split

# Split the data into training and testing sets
X = df.drop('parity', axis=1)
y = df['parity']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

# Define the neural network model
model = Sequential([
    Dense(2000, input_dim=8, activation='relu'), # Input layer with 2000 neurons and hidden layer with 16 neurons
    Dense(1000, activation='relu'), # Hidden layer with 1000 neurons
    Dense(1, activation='softmax') # Output layer with 1 neuron (parity bit)
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=16, validation_split=0.2, verbose=1)

# Function to predict and verify parity
def predict_parity(data):
    binary = np.array(list(map(int, format(data, '08b')))).reshape(1, -1)
    predicted_parity = (model.predict(binary) > 0.5).astype("int32")
    actual_parity = even_parity_bit(binary[0])
    print(f"Input Data: {format(data, '08b')}")
    print(f"Predicted Parity: {predicted_parity[0][0]}")
    print(f"Actual Parity: {actual_parity}")
    print("Verification:", "Pass" if predicted_parity[0][0] == actual_parity else "Fail")

# Example: Predict and verify parity for a new 8-bit input
predict_parity(100) # Example input: 100 (binary: 01100100)

```

Fig. 7. A.2. Script Contains the code for implementing and evaluating the ANN using TensorFlow, Scikit-learn, and Keras